

Risposte Parte B

Domanda 1

La gestione della memoria paginata in un sistema di elaborazione monoprocesso: se ne dia la definizione e se ne presenti la realizzazione e la gestione, evidenziandone caratteristiche, vantaggi e limiti.

1. Definizione

La paginazione è uno schema di gestione della memoria che elimina la necessità di allocare la memoria fisica in modo contiguo.

Il metodo suddivide la memoria in blocchi di dimensione fissa (solitamente potenze di 2, gestite dall'hardware):

- **Frame (Pagine Fisiche):** La memoria fisica è divisa in blocchi di dimensione fissa.
- **Pagine (Pagine Logiche):** Lo spazio di indirizzamento logico del processo è diviso in blocchi della stessa dimensione dei frame.

Questo disaccoppiamento permette all'utente di vedere una memoria continua, mentre fisicamente il processo può essere sparso in frame non contigui.

2. Realizzazione Architeturale (Hardware)

Il supporto hardware è fondamentale per la traduzione efficiente degli indirizzi. La CPU genera indirizzi logici divisi in due parti: **numero di pagina (p)** e **offset (d)**.

- **Tabella delle Pagine (Page Table):** È la struttura dati fondamentale che mappa ogni pagina logica (p) nel corrispondente frame fisico (f).
- **PTBR (Page-Table Base Register):** Poiché la tabella delle pagine risiede in memoria centrale, è necessario un puntatore hardware per localizzarla velocemente. Il **PTBR** è un registro della CPU che punta all'indirizzo fisico di inizio della tabella delle pagine del processo corrente.
 - *Funzione:* Durante un *context switch*, il Sistema Operativo deve solo aggiornare il valore del PTBR per caricare la tabella del nuovo processo, rendendo il cambio di contesto molto rapido.
- **MMU (Memory Management Unit):** Esegue la traduzione a runtime. Usa il PTBR per trovare la tabella, legge il numero di frame corrispondente alla pagina p e lo combina con l'offset d per generare l'indirizzo fisico.
- **TLB (Translation Look-aside Buffer):** Per evitare il doppio accesso in memoria (uno per leggere la tabella, uno per il dato) che raddoppierebbe i tempi di esecuzione, si usa la TLB, una cache hardware associativa veloce. Se la traduzione è nella TLB (*hit*), l'accesso è immediato; altrimenti (*miss*), si legge dalla tabella in RAM.

3. Gestione da parte del Sistema Operativo

Il Sistema Operativo gestisce l'allocazione, la protezione e l'ottimizzazione delle strutture dati.

- **Frame Table:** Una struttura globale che traccia lo stato di ogni frame fisico (libero/occupato, proprietario).
- **Allocazione:** Per eseguire un processo di N pagine, il SO trova N frame liberi ovunque in memoria e aggiorna la tabella del processo.
- **Protezione e Condivisione:** Ogni voce della tabella ha bit di controllo (es. *valid-invalid bit* o *read-write*) per proteggere la memoria. La paginazione facilita la condivisione di codice (es. librerie) mappando pagine logiche di processi diversi sugli stessi frame fisici.

Gestione di Grandi Tabelle delle Pagine:

Poiché le moderne tabelle delle pagine possono diventare enormi (occupando troppa memoria contigua), si adottano tecniche avanzate come:

1. **Paginazione Gerarchica:** La tabella delle pagine viene a sua volta paginata (es. a due livelli), permettendo di caricare in memoria solo le parti della tabella necessarie.
2. **Hashing:** Si usa una tabella hash per memorizzare solo le pagine allocate, riducendo lo spazio occupato.
3. **Tabella Invertita:** Esiste una sola tabella globale per tutto il sistema (una voce per ogni frame fisico) invece che una per processo. Risparmia memoria ma rende più complessa la condivisione.

4. Vantaggi e Limiti

Vantaggi:

- **Nessuna Frammentazione Esterna:** Non è necessario avere memoria contigua; qualsiasi frame libero può essere assegnato, risolvendo il problema della compattazione.
- **Allocazione Semplificata:** La gestione della lista dei frame liberi è rapida ed efficiente.
- **Condivisione e Protezione:** È semplice implementare la protezione della memoria e la condivisione di risorse (es. *shared memory*).

Limiti (Svantaggi):

- **Frammentazione Interna:** Poiché l'allocazione è a blocchi fissi, l'ultima pagina di un processo è raramente piena, sprecando spazio all'interno del frame (mediamente mezzo frame per processo).
- **Overhead di Memoria:** Ogni processo richiede la propria tabella delle pagine. Senza tecniche come la paginazione gerarchica, questo costo spaziale può essere elevato.
- **Overhead di Tempo:** L'accesso alla tabella in memoria rallenta l'esecuzione. È indispensabile l'uso di hardware dedicato (TLB) per mantenere prestazioni accettabili.

Domanda 2

La gestione della memoria segmentata in un sistema di elaborazione monoprocesso: se ne dia la definizione e se ne presenti la realizzazione e la gestione, evidenziandone caratteristiche, vantaggi e limiti.

La segmentazione è uno schema di gestione della memoria che supporta direttamente la visione logica che il programmatore (o l'utente) ha della memoria, superando la rigidità di un array lineare fisico.

1. Definizione

Mentre la memoria fisica è vista dall'hardware come un array unidimensionale, il programmatore vede un programma come un insieme di segmenti logici di dimensione variabile, raggruppati per tipo. Ogni segmento ha un proprio scopo logico (es. il programma principale, lo stack, la tabella dei simboli). Questa suddivisione implementa la *tipizzazione del codice*, un fattore molto importante per il Sistema Operativo. Ogni segmento è identificato da un nome (o numero) e ha una lunghezza intrinseca.

2. Realizzazione architetturale (Supporto HW)

L'indirizzo logico in questo schema è una coppia ordinata $\langle s, d \rangle$, dove s è il numero del segmento e d è l'offset (lo spiazzamento) all'interno del segmento. Per tradurre questo indirizzo, l'hardware (la Memory Management Unit o MMU) si avvale della *tabella dei segmenti*.

Ogni voce della tabella corrisponde a un segmento e contiene due valori fondamentali:

- **Base (segment base):** l'indirizzo fisico di partenza dove il segmento risiede in memoria RAM.
- **Limite (segment limit):** la lunghezza esatta del segmento (necessaria data la variabilità delle dimensioni).

Il funzionamento della traduzione prevede che l'hardware verifichi che l'offset d sia legale (ovvero $d < limite$). Se la condizione è verificata, l'indirizzo fisico viene calcolato sommando la base all'offset. Se $d \geq limite$, si genera una trap (errore di sistema / segmentation fault) per indirizzamento illegale. Anche in questo caso, per evitare eccessivi cali di prestazioni dovuti ai doppi accessi in memoria, si fa uso di una TLB (Translation Look-aside Buffer) come cache hardware veloce.

3. Gestione, Protezione e Condivisione

Il sistema operativo gestisce la tabella dei segmenti e l'allocazione dinamica della memoria fisica.

- **Protezione:** Poiché i segmenti rappresentano unità semantiche ben definite (tipizzate), è naturale associarvi permessi specifici tramite bit di controllo nella tabella. Ad esempio, il segmento del codice può essere definito in sola esecuzione, mentre un segmento dati in lettura/scrittura.
- **Condivisione:** La segmentazione facilita enormemente la condivisione di risorse (es. librerie condivise) tra più processi. È sufficiente assegnare nella tabella dei segmenti lo stesso frame fisico a processi diversi, impostando opportuni bit di protezione per mantenere la coerenza.

4. Vantaggi e limiti

- **Vantaggi:**
 - *Modello Naturale e Tipizzazione:* Rispecchia la reale struttura logica del programma, facilitando programmazione, compilazione e linking.
 - *Protezione Granulare:* Consente di gestire permessi su entità logiche dotate di senso, diversamente dai blocchi a dimensione fissa usati nella paginazione pura.
- **Limiti:**

- *Frammentazione Esterna*: È il difetto principale. Allocando e rimuovendo segmenti di dimensioni variabili si creano piccoli "buchi" di memoria libera inutilizzabili.
- *Gestione Complessa e Overhead*: Richiede algoritmi di allocazione dinamica onerosi (First-Fit, Best-Fit) e costringe il SO a eseguire operazioni di compattazione della memoria (garbage collection) che impattano pesantemente sulle prestazioni.

5. L'evoluzione: Segmentazione con Paginazione

Per superare i limiti prestazionali legati alla frammentazione esterna, i sistemi moderni uniscono le due tecniche. Mantenendo la divisione in segmenti, e quindi preservando i vantaggi della tipizzazione e della condivisione del codice, si suddividono internamente i segmenti stessi in *pagine di dimensione fissa*. Tali pagine vengono caricate in frame fisici, eliminando di fatto il problema della frammentazione esterna ereditato dalla segmentazione pura. In questa soluzione ibrida, gli indirizzi logici diventano una tripla: numero di segmento s , pagina p all'interno del segmento, e offset d all'interno della pagina.

Domanda 3

La gestione della memoria virtuale in un sistema di elaborazione monoprocesso: se ne dia la definizione e se ne presenti la realizzazione e la gestione, evidenziandone caratteristiche, vantaggi e limiti.

1. Definizione e Concetti Base

La memoria virtuale è un insieme di politiche e meccanismi che separa la memoria logica vista dal programmatore dalla memoria fisica realmente disponibile. Questo disaccoppiamento permette l'esecuzione di processi la cui dimensione logica è maggiore della RAM fisica e consente di caricare in memoria solo le porzioni di processo strettamente necessarie. Il programmatore è così libero di scrivere il codice ignorando i limiti fisici della memoria centrale.

2. Realizzazione Architeturale (Paginazione su Richiesta)

La tecnica standard per realizzare la memoria virtuale è la Paginazione su Richiesta (Demand Paging), guidata dal principio che una pagina viene caricata in memoria centrale solo quando è esplicitamente richiesta dall'esecuzione.

Il supporto hardware fondamentale include:

- **Bit di Validità**: Ogni voce nella tabella delle pagine ha un bit "Valid/Invalid". Se il bit è "Valid", la pagina è in RAM; se è "Invalid", la pagina si trova nella memoria secondaria.
- **Swap Space**: Un'area riservata su disco usata per conservare le pagine che non sono attualmente caricate in RAM.

3. Gestione Operativa (Page Fault e Sostituzione)

La gestione della memoria virtuale ruota attorno alla risoluzione delle mancanze di pagina e all'allocazione dei frame:

- **Gestione del Page Fault**: Se un processo tenta di accedere a una pagina marcata come "Invalid", la MMU genera un'interruzione (Page Fault). Il Sistema Operativo interviene cercando un frame libero in cui caricare la pagina richiesta dal disco. Se non ci sono frame liberi, il SO esegue un algoritmo di sostituzione per scegliere un frame "vittima" da scaricare. Infine, aggiorna la tabella delle pagine e riavvia l'istruzione interrotta.
- **Algoritmi di Sostituzione delle Pagine**: L'obiettivo è minimizzare il tasso di page fault. Le tecniche principali sono:
 - *FIFO*: Sostituisce la pagina in memoria da più tempo. Semplice, ma soffre dell'Anomalia di Belady (aumentando i frame disponibili può paradossalmente aumentare il numero di fault).
 - *Ottimale (OPT)*: Benchmark teorico che sostituisce la pagina che non verrà usata per il tempo più lungo.
 - *LRU (Least Recently Used)*: Sostituisce la pagina inutilizzata da più tempo. È molto efficiente ma richiede un costoso supporto hardware (es. timestamp o stack).
 - *Approssimazioni (Seconda Chance/Clock)*: Usano un "bit di riferimento" hardware. Se è 1, viene azzerato (dando una seconda possibilità); se è 0, la pagina viene sostituita.
- **Allocazione dei Frame**: Il SO deve decidere quanti frame assegnare a ciascun processo. L'allocazione può essere *Omogenea* (uguale per tutti) o *Proporzionale* (basata su dimensione o priorità). Inoltre, la sostituzione della vittima può essere *Globale* (tra tutti i processi) o *Locale* (solo tra i frame del processo stesso).

4. Thrashing e Tecniche di Prevenzione

Il limite prestazionale più grave della memoria virtuale è il **Thrashing**: se un processo non possiede abbastanza frame allocati, continuerà a generare page fault, portando il sistema a spendere più tempo nelle operazioni di paginazione (I/O su disco) che nell'esecuzione effettiva delle istruzioni.

Per evitare il thrashing, il SO sfrutta il principio di località e interviene sulla multiprogrammazione tramite due modelli principali:

- **Modello Working Set:** Si definisce una "finestra" (Working Set) che traccia le pagine a cui il processo ha acceduto negli ultimi N accessi. Se la somma delle dimensioni dei working set attivi di tutti i processi supera i frame fisici disponibili, il sistema è in thrashing e il SO sospende temporaneamente uno o più processi, liberando i loro frame. Questa stima è utile anche per la "pre-paginazione" al ripristino di un processo.
- **Strategia Page-Fault Frequency (PFF):** Il SO controlla direttamente il tasso di page fault dei processi. Vengono impostate una soglia massima e una minima: se i fault superano la soglia massima, al processo viene assegnato un nuovo frame; se scendono sotto la soglia minima, gliene viene tolto uno. Se serve un frame ma non ce ne sono di liberi, un processo viene sospeso.

5. Vantaggi e Limiti Generali

- **Vantaggi:**
 - *Disaccoppiamento e Flessibilità:* Lo spazio logico può superare largamente la memoria fisica, rendendo i programmi indipendenti dalla configurazione hardware.
 - *Multiprogrammazione Elevata:* Caricando solo frammenti necessari, più processi possono risiedere in RAM contemporaneamente, migliorando l'utilizzo della CPU.
 - *Efficienza di I/O:* Meno operazioni inutili di disco, poiché si caricano solo i dati e il codice realmente usati (es. routine per errori rari mai caricate).
- **Limiti:**
 - *Complessità Hardware e Software:* Richiede una stretta integrazione tra meccanismi HW (MMU, TLB) e algoritmi complessi nel kernel.
 - *Overhead Prestazionale:* Gestire un page fault impiega tempi di I/O (millisecondi) ordini di grandezza più lenti rispetto all'esecuzione in RAM.
 - *Rischio di Thrashing:* Come discusso, una cattiva gestione dell'allocazione o un grado di multiprogrammazione eccessivo possono degradare drammaticamente le prestazioni.

Domanda 4

La gestione delle periferiche in un sistema di elaborazione monoprocesso: si descriva l'organizzazione del software di sistema per la loro gestione e le principali funzioni che esso deve realizzare, evidenziandone caratteristiche, vantaggi e limiti.

La gestione dell'Input/Output (I/O) è l'aspetto del sistema operativo (SO) che si occupa di gestire l'estrema varietà di dispositivi periferici. Il suo obiettivo principale è astrarre la complessità hardware per offrire alle applicazioni un'interfaccia semplice, uniforme e indipendente dal dispositivo fisico.

1. Caratteristiche delle periferiche

Le periferiche sono molteplici e vengono classificate in base a caratteristiche chiave che ne dettano le logiche di gestione:

- **Modalità di trasferimento:** A blocchi (i dati sono scambiati in blocchi di dimensione fissa, es. dischi) o a caratteri (flusso continuo di byte, es. tastiere).
- **Metodo di accesso:** Sequenziale (accesso in un ordine fisso prefissato, es. nastri) o Diretto/Casuale (accesso immediato a qualsiasi blocco, es. dischi).
- **Condivisione:** Dedicate (usate da un solo processo alla volta) o Condivisibili (gestiscono richieste concorrenti da più processi).
- **Direzione I/O:** Dispositivi di sola lettura, sola scrittura, o lettura/scrittura.
- **Velocità:** Varia di ordini di grandezza, influenzando i tempi di trasferimento, di ricerca e la latenza.
- **Schedulazione del trasferimento:** Può essere sincrona (blocca il processo finché l'operazione non termina) o asincrona (il processo continua la sua esecuzione parallelamente all'operazione di I/O).

2. Organizzazione del Software (Struttura a strati)

Il software di I/O è organizzato in livelli gerarchici, dove ogni livello maschera i dettagli di quello inferiore per semplificarne l'utilizzo:

- **Gestione del Canale e Interruzioni (Hardware):** Livello più basso, che nasconde la gestione della connessione fisica e gestisce le interruzioni generate al termine di un'operazione.
- **Device Dependent Drivers (DDD):** Programmi specifici per ogni dispositivo, spesso scritti dai produttori. Incapsulano i dettagli tecnici hardware e traducono le richieste generiche in comandi specifici per il controller.

- **Device Independent Drivers (DID) / Sottosistema I/O:** Il livello del kernel che esegue funzioni comuni a tutti i dispositivi offrendo un'interfaccia standardizzata e uniforme ai livelli superiori.
- **Software Utente:** Include le librerie di sistema (API) usate dai processi e demoni di sistema.

3. Funzioni del sottosistema I/O:

Il livello "indipendente dai dispositivi" implementa meccanismi chiave per prestazioni e concorrenza:

- **Schedulazione dell'I/O:** Riordina le richieste in coda per migliorare l'efficienza e l'equità. Ad esempio, nel disco, le richieste vengono ordinate per minimizzare il movimento meccanico della testina, riducendo i tempi di attesa rispetto all'ordine di arrivo (FCFS).
- **Bufferizzazione:** Utilizza aree di memoria (buffer) per tre motivi:
 1. Compensare la differenza di velocità tra produttore (CPU) e consumatore (dispositivo).
 2. Adattare dispositivi con diverse dimensioni di trasferimento.
 3. Supportare la "semantica della copia" (garantire che i dati scritti su disco siano quelli presenti al momento della chiamata, anche se l'applicazione li modifica subito dopo).
- **Caching:** Mantiene in una memoria veloce una copia dei dati letti di recente dalle periferiche, per consentire un riuso rapido ed evitare lenti accessi ripetuti.
- **Spooling e Locking:** Gestisce l'accesso alle periferiche dedicate e lente. Lo spooling devia l'output su un file su disco gestito da un demone (spooler) per non bloccare il processo, mentre il locking garantisce l'accesso in mutua esclusione.
- **Gestione Errori:** Il SO gestisce in autonomia gli errori temporanei (es. rilettura del disco) e comunica ai processi i codici di errore in caso di guasti hardware permanenti.

4. Vantaggi e limiti:

- **Vantaggi:**
 - *Astrazione e Uniformità:* I programmatori utilizzano interfacce logiche omogenee (es. read/write) indipendentemente dal dispositivo fisico.
 - *Modularità:* È possibile aggiungere nuove periferiche installando semplicemente i driver (DDD) senza dover modificare il kernel.
 - *Efficienza:* Tecniche come buffering, caching e DMA (Direct Memory Access) mascherano la lentezza delle periferiche rispetto alla CPU.
- **Svantaggi (Limiti):**
 - *Overhead Prestazionale:* Il passaggio attraverso i vari strati software e la copia continua dei dati tra spazio utente e kernel consumano risorse, cicli di CPU e banda di memoria.
 - *Gestione Interruzioni:* Un carico eccessivo di I/O causa frequenti interrupt e cambi di contesto che possono degradare le prestazioni (es. livelock in casi estremi di rete).
 - *Complessità e Sicurezza:* I device driver operano ad alto livello di privilegio; bug o driver mal scritti possono causare il crash dell'intero sistema operativo.

Domanda 5

Il file system in un sistema di elaborazione monoprocesso: si descrivano il concetto di file e di file system e le loro principali realizzazioni, evidenziandone caratteristiche, vantaggi e limiti.

Il File System è la componente del sistema operativo che gestisce le informazioni sulle memorie di massa (dischi), astraendo dalla visione fisica dei dispositivi per fornire un'interfaccia logica, omogenea e persistente. Il File System è tipicamente implementato a strati (gestione periferica, FS di base, organizzazione logica)

1. Concetti Base: File e Descrittore

- **Il File (Unità Logica):** È l'unità logica di memorizzazione. A livello logico è visto come una sequenza (byte stream) di elementi omogenei, il cui significato (testo, eseguibile, immagine) è imposto dalle applicazioni che lo interpretano, non dal SO.
- **Il Descrittore:** Ogni file è associato a un descrittore contenente metadati essenziali, come nome, ID, tipo, dimensione, permessi di protezione e timestamp di creazione/modifica.
- **Operazioni Base:** Il SO gestisce la creazione, lettura, scrittura, posizionamento (seek) e cancellazione dei file.

2. Organizzazione Logica (Le Directory)

Per evitare che tutti i file siano aggregati in un unico spazio caotico, i file system utilizzano le Directory.

- **Astrazione:** Una directory è una struttura dati che contiene riferimenti ad altre directory o ai descrittori dei file.
- **Struttura:** Il modello standard è ad **albero**, con una radice che scende verso foglie annidate, risolvendo i problemi di omonimia (file con lo stesso nome in cartelle diverse). Modelli più avanzati utilizzano **grafi aciclici** o generali, permettendo a un file di essere condiviso ed esplorato da più directory contemporaneamente.
- **Implementazione:** Le directory possono essere realizzate con semplici *liste* (ricerca lineare lenta) o con *tabelle di hash* (ricerca rapidissima, ma che richiede la gestione delle collisioni).

3. Realizzazione Fisica (Metodi di Allocazione)

Il SO deve decidere come mappare i blocchi logici del file sui blocchi fisici del disco.

- a. **Allocazione Contigua:** Il file occupa blocchi fisici adiacenti.
 - *Vantaggi:* Accesso sequenziale e diretto eccellente.
 - *Svantaggi:* Causa frammentazione esterna e rende difficile l'espansione dei file. Si può mitigare usando gli *extent* (frammenti aggiuntivi collegati).
- b. **Allocazione Collegata (Linked):** I blocchi sono sparsi nel disco; ogni blocco contiene i dati e un puntatore al blocco successivo (lista concatenata).
 - *Vantaggi:* Risolve la frammentazione esterna; il file può crescere facilmente.
 - *Contro:* Accesso casuale lentissimo e spazio sprecato per i puntatori. Una variante efficiente è la FAT (File Allocation Table), tenuta interamente in memoria centrale.
- c. **Allocazione Indicizzata:** Tutti i puntatori ai blocchi dati sono raccolti in un unico blocco speciale chiamato **Blocco Indice**.
 - *Pro:* Supporta l'accesso diretto veloce senza frammentazione esterna.
 - *Contro:* Overhead di memoria per file piccoli (si spreca un intero blocco solo per l'indice).
- d. **Soluzione Ibrida (Es. UNIX Inode):** Combina puntatori diretti per i primi blocchi di dati (accesso immediato) e puntatori indiretti (che puntano ad altri blocchi indice) per scalare dinamicamente con file molto grandi.

4. Gestione dello spazio libero

Per evitare sprechi, il File System deve tracciare con efficienza i blocchi cancellati.

- **Bitmap:** Un array di bit dove 1 indica un blocco libero e 0 occupato. Eccellente per trovare blocchi contigui, ma la mappa deve risiedere in RAM per essere veloce.
- **Lista Collegata:** Ogni blocco vuoto punta al successivo blocco vuoto. Facile trovare un singolo blocco, difficile trovarne N contigui.
- **Conteggio:** Sfrutta la contiguità. Memorizza l'indirizzo di un blocco libero e il numero di blocchi liberi che lo seguono immediatamente.

5. Vantaggi e Limiti generali

- **Vantaggi:** Il file system offre un'astrazione potente, mascherando all'utente la complessa geometria hardware. Le prestazioni sono ulteriormente migliorabili implementando meccanismi di *caching* (es. algoritmi LRU, read-ahead per anticipare le letture) per velocizzare gli accessi ai dischi.
- **Limiti:** Oltre ai problemi fisici di *frammentazione* interna (blocchi allocati non riempiti del tutto) ed esterna che degradano le prestazioni nel tempo, il file system è vulnerabile ai crash. Interruzioni improvvise durante scritture critiche (es. aggiornamento directory) causano inconsistenze, che richiedono l'uso di software di ripristino (fsck) o architetture più robuste basate su *Journaling*.

Domanda 6

La protezione dei file in un sistema di elaborazione monoprocesso: si descrivano il concetto di protezione dei file e sue principali realizzazioni, evidenziandone caratteristiche, vantaggi e limiti.

1. Concetti Fondamentali: Regole, Meccanismi e Minimo Privilegio

Il sistema di protezione del Sistema Operativo serve a prevenire accessi non autorizzati o accidentali alle risorse. Per farlo, separa nettamente due aspetti:

- **Regole (Politiche):** Stabiliscono *chi* può accedere a una risorsa e *in che modo*.
- **Meccanismi:** Sono gli strumenti hardware e software che impongono il rispetto di queste regole.

Il principio cardine è il **Principio della Minima Conoscenza (o Minimo Privilegio)**: un processo deve operare esclusivamente con i diritti strettamente necessari al suo scopo, limitando così i danni in caso di errori o vulnerabilità.

2. Domini di Protezione

Per gestire le autorizzazioni, ogni processo opera all'interno di un **Dominio di Protezione**, ovvero l'insieme delle risorse a cui il processo può accedere e delle relative operazioni lecite. Un diritto è tipicamente rappresentato dalla coppia `<Risorsa, Operazioni>`.

L'associazione tra un processo e un dominio può essere:

- **Statica:** Il processo entra in un dominio all'avvio e vi rimane fino alla terminazione. Questo approccio è rigido e spesso costringe a fornire più permessi del necessario.
- **Dinamica:** Il processo può effettuare un cambio di dominio (switch) durante la sua esecuzione, aderendo meglio al principio del minimo privilegio

3. La Matrice degli Accessi

Il modello astratto per rappresentare le politiche di protezione è la Matrice degli Accessi.

- **Righe:** Rappresentano i Domini (utenti o processi).
- **Colonne:** Rappresentano gli Oggetti/Risorse.
- **Celle:** Contengono i permessi (es. Read, Write)

Per supportare i sistemi dinamici, la matrice può includere i domini stessi come colonne: se nella cella compare l'operazione di *switch*, il processo può cambiare dominio. Possono inoltre essere definiti diritti di *copia* (per propagare permessi) o di *proprietà* (per modificare i permessi altrui su una risorsa propria).

4. Implementazioni Pratiche

Poiché una matrice completa sarebbe enorme e piena di celle vuote (sparsa), causando sprechi di memoria e inefficienza, si utilizzano approcci alternativi:

- **Tabella Globale:** Consiste in un'unica grande tabella che memorizza solo le celle piene della matrice, salvandole sotto forma di triplette `<Dominio, Risorsa, Diritti>`. *Limiti:* È un approccio inefficiente. La tabella rischia comunque di diventare gigantesca; inoltre, dovendo essere tenuta in memoria centrale per eseguire le verifiche ad ogni accesso, richiederebbe lo spreco di svariate pagine di memoria.
- **Liste di Controllo degli Accessi (ACL):** La matrice viene decomposta per colonne. A ogni risorsa è associata una lista delle coppie `<Dominio, Diritti>`. *Vantaggi:* Gestione fine dei permessi sui singoli file e facilità nel sapere chi accede a una risorsa. *Limiti:* È inefficiente per determinare tutti i permessi posseduti da un singolo utente/dominio.
- **Liste delle Capacità (C-Lists):** La matrice è decomposta per righe. Ogni dominio possiede una lista (o "biglietto") di coppie `<Risorsa, Diritti>`. *Vantaggi:* Verifica dell'accesso immediata ed efficiente. *Limiti:* È molto difficile gestire la revoca dei permessi.
- **Meccanismo Serratura-Chiave (Lock-Key):** Compromesso ibrido. Ogni risorsa possiede liste di pattern di bit (*Lock* o serrature), mentre ogni processo ha delle chiavi (*Key*). L'accesso avviene solo se la chiave del processo combacia con la serratura della risorsa.

5. La Revoca dei Diritti

In un sistema dinamico è cruciale poter revocare i permessi concessi. La revoca può essere immediata o ritardata, parziale o totale, temporanea o permanente.

- Nelle **ACL** la revoca è semplice: basta rimuovere la voce dalla lista associata al file.
- Nelle **C-Lists**, poiché le informazioni sono distribuite tra i processi, si adottano soluzioni come la *riacquisizione* (le liste vengono svuotate periodicamente costringendo i processi a richiederle, momento in cui possono essere negate) oppure l'uso di *puntatori indiretti* centralizzati che rimandano alle capability (per annullarle basta eliminare il puntatore).

6. Vantaggi e Limiti Generali

- **Vantaggi:** Garantisce isolamento impedendo a processi difettosi di danneggiare il SO o altri processi. Assicura l'integrità dei dati e previene accessi non autorizzati.
- **Limiti:** Introduce un overhead prestazionale, in quanto ogni accesso a una risorsa richiede il controllo dei permessi consumando cicli di CPU. Inoltre, la gestione complessa dei gruppi e della revoca dinamica può introdurre vulnerabilità se non configurata correttamente.